DjangoSentinel Developer Manual

Architecture, extension points, and contribution guide

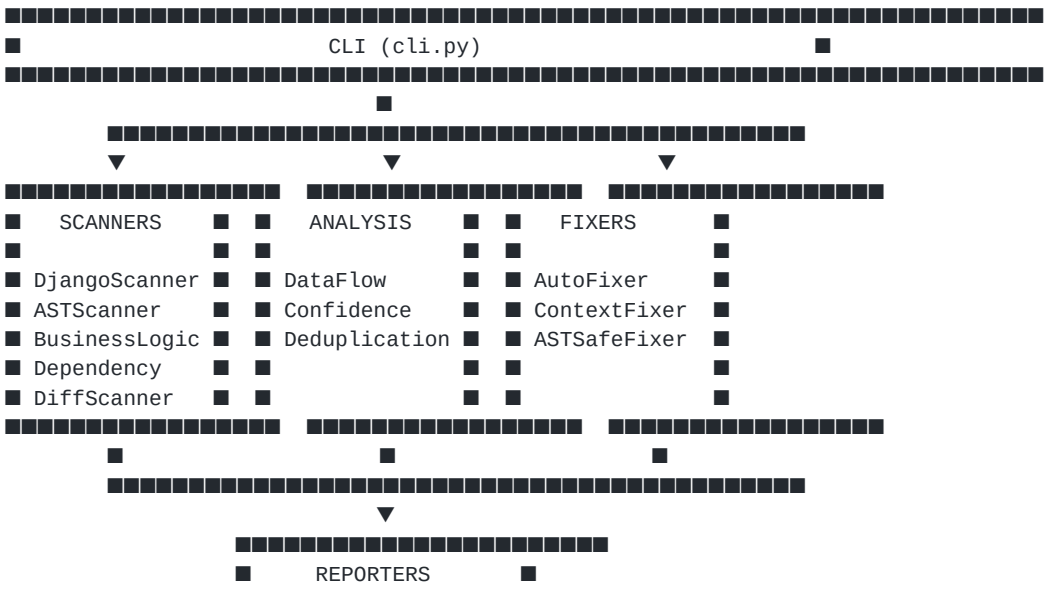
DjangoSentinel v1.0.0 — Developer Manual

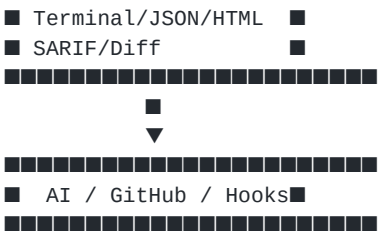
Table of Contents

1. [Architecture Overview](#architecture-overview)
2. [Project Structure](#project-structure)
3. [Core Data Models](#core-data-models)
4. [Scanner Development](#scanner-development)
5. [Fixer Development](#fixer-development)
6. [Reporter Development](#reporter-development)
7. [Analysis Engine](#analysis-engine)
8. [AI Engine](#ai-engine)
9. [GitHub Integration](#github-integration)
10. [Cache System](#cache-system)
11. [Testing](#testing)
12. [Contributing](#contributing)

Architecture Overview

DjangoSentinel is a multi-engine security scanner built on a pipeline architecture:





Engine Layers:

1. **Rule Engine** — regex + AST patterns (always runs)
2. **AST Engine** — structural code analysis (always runs)
3. **Taint Engine** — data-flow source→sink tracking (always runs)
4. **AI Engine** — Claude-powered fixes/explanations (`--ai`)
5. **Diff Engine** — git-diff scoped scanning (`--diff`)
6. **Cache Engine** — file-hash incremental scanning (automatic)

Project Structure

```
django-sentinel/
├── djangosentinel/
│   ├── __init__.py          # Package exports & version
│   ├── __main__.py          # python -m entry point
│   ├── cli.py               # CLI parser & command handlers
│   ├── models.py            # Core data models
│   ├── cache.py             # Scan cache & incremental scanner
│   ├── ai_engine.py         # Claude AI integration
│   ├── github_integration.py # GitHub PR API
│   ├── scanners/
│   │   ├── __init__.py
│   │   ├── django_scanner.py         # Django-specific patterns
│   │   ├── ast_scanner.py            # AST structural analysis
│   │   ├── business_logic_scanner.py # Logic flaws & IDOR
│   │   ├── dependency_scanner.py     # CVE database
│   │   └── diff_scanner.py           # Diff parsing & scoped scan
│   ├── analysis/
│   │   ├── __init__.py
│   │   ├── data_flow.py      # Taint tracking
│   │   └── confidence.py     # Scoring & deduplication
│   ├── fixers/
│   │   ├── __init__.py
│   │   ├── auto_fixer.py     # Rule-based fixer
│   │   ├── context_fixer.py  # Context-aware fixer
│   │   └── ast_safe_fixer.py # Indentation-preserving fixer
│   ├── reporters/
│   │   ├── __init__.py
│   │   └── reporters.py      # All output formats
│   └── hooks/
│       ├── __init__.py
│       └── git_hooks.py      # Git hook installer
├── tests/
│   └── test_all.py           # 104-test suite
├── docs/                     # Additional documentation
├── setup.py                  # Package metadata
└── pyproject.toml            # Build & tool config
```

Core Data Models

Located in `djangosentinel/models.py`.

`Severity`

Enum: `CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, `INFO`

`VulnCategory`

Enum: `INJECTION`, `XSS`, `CSRF`, `AUTH`, `SENSITIVE_DATA`, `MISCONFIG`, `INSECURE_DEPS`, `BUSINESS_LOGIC`, `IDOR`, `SSRF`, `XXE`, `OPEN_REDIRECT`, `RACE_CONDITION`, `MASS_ASSIGNMENT`, `HARDCODED_SECRET`

`CodeLocation`

```
@dataclass
class CodeLocation:
    file_path: str
    line_number: int
    column: Optional[int] = None
    end_line: Optional[int] = None
    snippet: Optional[str] = None
```

`Fix`

```
@dataclass
class Fix:
    description: str
    original_code: str
    fixed_code: str
    applied: bool = False
    manual_review_required: bool = False
    explanation: str = ""
```

`Vulnerability`

```
@dataclass
class Vulnerability:
    vuln_id: str
    title: str
    category: VulnCategory
    severity: Severity
    location: CodeLocation
    description: str
    owasp_ref: str = ""
    cwe_id: str = ""
    cve_id: str = ""
    fix: Optional[Fix] = None
    false_positive: bool = False
    references: List[str] = field(default_factory=list)
    metadata: Dict[str, Any] = field(default_factory=dict)
```

Key properties: `confidence`, `confidence_label`, `is_taint_detected`, `is_ai_detected`, `is_ast_detected`

`ScanResult`

Aggregates findings with counts: `critical_count`, `high_count`, `medium_count`, `low_count`, `fixable_count`, `fixed_count`, and `summary()`.

Scanner Development

All scanners follow the same interface:

```
class MyScanner:
    def __init__(self, project_path: str):
        self.project_path = Path(project_path)
        self.vulnerabilities = []

    def scan(self) -> List[Vulnerability]:
        # Walk files, detect patterns, return findings
        return self.vulnerabilities
```

Adding a Scanner

1. Create `djangosentinel/scanners/my_scanner.py`
2. Implement `__init__(self, project_path)` and `scan() -> List[Vulnerability]`
3. **Always set** `metadata["source"]` — used by confidence scoring:

   ```
   v.metadata["source"] = "my_scanner"
   ```

4. Export it in `scanners/__init__.py`
5. Register it in `cli.py` `run_full_scan()`:

   ```
   scanners_to_run = [
       ("■ Django Scanner", lambda: DjangoScanner(project_path).scan(), True),
       ("■ My Scanner",     lambda: MyScanner(project_path).scan(),     True),
   ]
   ```

Base confidence values (`analysis/confidence.py`)

```
BASE_CONFIDENCE = {
    "ast_scanner":        0.82,
    "taint_tracker":      0.90,
    "django_scanner":     0.75,
    "business_logic":     0.70,
    "dependency_scanner": 0.95,
    "ai_pattern_scanner": 0.65,
    "diff_scanner":       0.75,
    "unknown":            0.60,
}
```

AST Scanner Example

Use `ast.parse()` and `ast.NodeVisitor`. See `scanners/ast_scanner.py` for the visitor pattern with parent tracking.

Diff Scanner

`DiffScanner` wraps other scanners and filters findings to changed lines only. See `DiffParser` for unified diff parsing (`all_added_lines`, `hunks`, `is_new_file`).

Fixer Development

`AutoFixer`

Rule-based fixer that delegates to `ASTSafeFixer`. Creates backups in `.sentinel_backups/`.

`ContextAwareFixer`

Combines rule-based fixes with AI. Categories:

1. Rule-fixable (`fix` present, `manual_review_required=False`, `fixed_code` not starting with `#`)
2. AI-needed (`manual_review_required=True` or placeholder hint)

3. Manual (no fix)

`ASTSafeFixer`

The core fix application engine. Four-stage approach:

1. **Exact inline replacement** — single line, simple swap
2. **Line-number targeted block replacement** — multi-line with indentation preserved
3. **Pattern-based replacement** — using `location.snippet`
4. **Whole-source replacement** — last resort

Critical invariants:

- Parse source with `ast.parse()` before touching it
- Validate `ast.parse(new_source)` after the fix — abort if syntax breaks
- Detect and preserve target line indentation via `detect_indent()` / `apply_indent()`
- Never apply fixes with `manual_review_required=True`

Adding a rule-based fix: add `fix_original` / `fix_replacement` to a check dict in `DjangoScanner.SETTINGS_CHECKS`.

Reporter Development

All reporters implement `report(self, result: ScanResult) -> str`.

```
class MyReporter:
    def report(self, result: ScanResult) -> str:
        return "..."
```

Register in `cli.py` `cmd_scan()` and export in `reporters/__init__.py`.

`DiffReporter`

Specialised for `--diff` mode. Takes `diff_summary` dict and implements both `report()` (terminal) and `report_json()`.

`SARIFReporter`

Produces SARIF 2.1.0 for GitHub Code Scanning. Maps severity to SARIF levels (`error`/`warning`/`note`).

Analysis Engine

Data-Flow Taint Tracking (`analysis/data_flow.py`)

```
tracker = DataFlowTracker(project_path)
flows = tracker.analyze_project()   # all files
flows = tracker.analyze_file("views.py", source)  # single file
```

Concepts:

- `TAINT_SOURCES` — where user data enters (`request.GET`, `request.POST`, etc.)
- `SINKS` — dangerous operations (dict: pattern → `{vuln, cwe, severity}`)
- `SANITIZERS` — functions that clean tainted data (`int()`, `escape()`, `get_object_or_404`)
- `_TaintVisitor` — AST visitor tracking taint propagation

- `DataFlow` dataclass — confirmed source→sink flow

Adding a sink: add to the `SINKS` dict:

```
"my_dangerous_call": {"vuln": "My Vuln", "cwe": "CWE-XXX", "severity": "HIGH"},
```

Confidence Scoring (`analysis/confidence.py`)

- `compute_confidence(vuln, source_line)` — base score + severity boost + signal checks
- `deduplicate(vulns)` — merges findings by `finding_hash` (file + line bucket + category + CWE)
- `apply_confidence_scores(vulns, source_map)` — bulk scoring
- `filter_by_confidence(vulns, min_confidence)` — threshold filter
- `score_label` / `score_emoji` — display helpers

Deduplication: keeps highest-confidence finding; merges references, metadata, and adopts a fix from duplicates if the winner lacks one.

AI Engine

Located in `ai_engine.py`. Pure HTTP (no SDK dependency).

```
_call_claude(system, user, max_tokens, api_key, timeout) -> Optional[str]
```

Classes:

- `AIFixEngine` — generates fixes for `manual_review_required` vulns
- `AIExplainer` — developer-friendly explanations
- `AIPatternScanner` — reasons about novel patterns (`--ai-deep`)
- `PRReviewSummarizer` — GitHub PR markdown summaries

All AI classes expose `.available` (True if `ANTHROPIC_API_KEY` set) and are no-ops when unavailable.

Adding an AI capability: add a class with a `SYSTEM_PROMPT`, call `_call_claude()`, parse the response, and return a `Fix` / `str` / `List[Vulnerability]`.

GitHub Integration

Located in `github_integration.py`.

```
gh = GitHubPRIntegration(token=..., repo=..., pr_number=..., commit_sha=...)
gh.available                      # bool
gh.post_summary_comment(result, summarizer)
gh.post_inline_review(result)     # -> count
gh.create_check_run(result)       # -> bool
gh.get_pr_changed_files()         # -> List[str]
```

Environment variables auto-detected: `GITHUB_TOKEN`, `GITHUB_REPOSITORY`, `GITHUB_SHA`, `GITHUB_REF`. PR number parsed from `refs/pull/N/merge`.

Cache System

Located in `cache.py`.

- `file_hash(path)` — SHA-256 (first 16 chars)

- `ScanCache` — persistent JSON cache at `.sentinel_cache/scan_cache.json`
- `IncrementalScanner` — wraps any scanner with cache hits/misses

Cache rules:

- Only caches findings with `confidence >= 0.5`
- Never caches AI findings (title starts with `[AI]`)
- Invalidates entire cache on `SENTINEL_VERSION` change
- TTL: 7 days
- Conservative: cache-miss means re-scan

Adding cache invalidation: bump `SENTINEL_VERSION` in `cache.py`.

Testing

Run the full suite:

```
python -m pytest tests/test_all.py -v
# or
python tests/test_all.py
```

104 tests across 12 classes:

Class	Tests	Coverage
`TestDjangoScanner`	20	settings, views, templates, secrets
`TestASTScanner`	10	structural detections
`TestBusinessLogicScanner`	12	logic flaws + AST IDOR
`TestDependencyScanner`	6	CVE database, imports
`TestDiffParser`	8	unified diff parsing
`TestDiffScanner`	9	diff-scoped detection
`TestDataFlowTracker`	8	taint analysis
`TestConfidenceScoring`	8	scoring + dedup
`TestASTSafeFixer`	8	fix application
`TestScanCache`	6	file-hash cache
`TestAutoFixer`	3	simple fixes
`TestReporters`	5	output formats

Writing a Test

Use `TempDirMixin` for temp project directories:

```
class TestMyScanner(TempDirMixin, unittest.TestCase):
    def scan(self):
        return MyScanner(self.test_dir).scan()

    def test_detects_something(self):
        self.write_file("views.py", "dangerous_code()\n")
```

```
        self.assertTrue(any("Vuln" in v.title for v in self.scan()))
```

AI Model Provider Abstraction

DjangoSentinel uses a model provider abstraction layer defined in `djangosentinel.ai_model_providers`. This layer provides a uniform interface over multiple LLM providers while maintaining full backward compatibility.

Provider Classes:

- `AnthropicProvider` — wraps Anthropic Claude API
- `OpenAIProvider` — wraps OpenAI GPT API
- `GeminiProvider` — wraps Google Gemini API

Base Class: `AIModelProvider`

Each provider implements:

- `_get_api_key()` — retrieves API key from environment
- `_call_api(system, user, max_tokens, timeout)` — makes the API call
- `generate(system, user, max_tokens, timeout)` — public entry point

Global Provider Management:

- `get_ai_provider()` — returns the current default provider
- `set_ai_provider(provider)` — sets a new default provider
- `is_ai_available()` — checks if the current provider has an API key

Default Behavior:

The default provider is `AnthropicProvider`, which preserves full backward compatibility.

Existing code using `_call_claude`, `AIFixEngine`, `AIExplainer`, etc. continues to work

without any modifications.

Adding a New Provider:

To add support for a new LLM provider:

1. Create a class that inherits from `AIModelProvider`
2. Implement `_get_api_key()` to read the appropriate environment variable
3. Implement `_call_api()` with the provider's HTTP API details
4. Optionally register the provider or make it selectable via `set_ai_provider()`

Example: Custom Provider

```
from djangosentinel.ai_model_providers import AIModelProvider

class CustomProvider(AIModelProvider):
    def _get_api_key(self):
        return os.environ.get("CUSTOM_API_KEY", "")

    def _call_api(self, system, user, max_tokens, timeout):
        # Custom API call implementation
        pass
```

CLI Integration:

All CLI commands that use AI (`--ai`, `--ai-deep`, `--ai-explain`) respect the

currently set provider. No code changes are needed to support multiple providers

— just set the appropriate environment variable.

Docker Containerized Deployment and Configuration

DjangoSentinel can be deployed inside Docker containers for isolated, reproducible scanning environments. This is particularly useful for CI/CD pipelines, development consistency, and production hardening.

Dockerfile

The project includes a `Dockerfile` that uses `python:3.12-slim` as the base image and installs `django-sentinel` from PyPI. The default command is `sentinel scan .`, which runs a full security scan when the container starts.

Key Dockerfile features:

- `pip install django-sentinel` (installs the package from PyPI)
- Environment variables passed at runtime via `docker run -e` or `docker-compose`
- Host network mode for GitHub API access
- Caching via `.anthropic` volume (in docker-compose)

Dockerfile contents:

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir django-sentinel
CMD ["sentinel", "scan", "."]
```

docker-compose.yml

The included `docker-compose.yml` defines a development service `sentinel` that:

- Mounts the current directory as a volume (`.:/app`) for code scanning
- Reads environment variables from `.env` file
- Uses the host's network stack for GitHub API access
- Optionally caches data in a `.anthropic` volume

docker-compose.yml service definition:

```
services:
  sentinel:
    build: .
    container_name: django-sentinel
    restart: unless-stopped
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - GOOGLE_API_KEY=${GOOGLE_API_KEY}
      - GITHUB_TOKEN=${GITHUB_TOKEN}
      - GITHUB_REPOSITORY=${GITHUB_REPOSITORY}
      - GITHUB_SHA=${GITHUB_SHA}
      - GITHUB_REF=${GITHUB_REF}
    volumes:
      - .:/app
    working_dir: /app
    network_mode: host
```

.env.example

The included `.env.example` file provides a template for all supported environment variables:

Variable	Description	Example
`ANTHROPIC_API_KEY`	Claude API key for AI features	`sk-ant-...`
`OPENAI_API_KEY`	OpenAI GPT-4/3.5 API key	`sk-...`
`GOOGLE_API_KEY`	Google Gemini API key	`AIza...`
`GITHUB_TOKEN`	GitHub Personal Access Token	`ghp_...`
`GITHUB_REPOSITORY`	Repository owner/name	`myorg/myapp`
`GITHUB_SHA`	Git commit SHA	`a1b2c3d4`
`GITHUB_REF`	Git reference (PR number)	`refs/pull/42/merge`

Usage:

```
cp .env.example .env
# Edit .env with your keys
docker-compose up --build
# Or run standalone:
docker run -it --rm -v $(pwd):/app -e ANTHROPIC_API_KEY="your-key" django-sentinel sentinel scan .
```

Developer Workflow with Docker

1. **Clone the repository**
2. **Copy** `.env.example` **to** `.env` and fill in your API keys
3. **Run** `docker-compose up --build` to start the scanner container
4. **View results** in the container output or mounted report files
5. **Modify code** - changes are reflected instantly via the volume mount
6. **Run** `docker-compose down` to stop the container

CI/CD Integration

For GitHub Actions or other CI systems, use the environment variables directly without Docker compose:

```
- name: DjangoSentinel Security Scan
  run: sentinel scan . --format sarif -o report.sarif
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/my-scanner`)
3. Add your code following existing patterns
4. Add tests (maintain 104+ passing)
5. Run `python -m pytest tests/test_all.py -v`
6. Ensure no regressions

7. Submit a pull request

Code Style

- No comments unless they explain non-obvious logic
- Type hints on public functions
- Dataclasses for data structures
- Emoji icons in terminal output for user-friendliness
- Errors caught gracefully (`except Exception: continue`) in scanners to never crash a scan

Architecture Principles

1. **Scanners are dumb** — detect and return, no side effects
2. **Analysis is pure** — scoring/dedup take lists, return lists
3. **Fixers are safe** — validate before and after; never corrupt code
4. **Reporters are pure** — take `ScanResult`, return string
5. **AI is optional** — every AI feature has a non-AI fallback

---

Version History

- **1.0.0** — Initial stable release: full scanner suite, taint analysis, auto-fixers, 5 reporters, AI engine, GitHub integration, git hooks, incremental cache.

License

MIT License — see LICENSE file for details.